

Verifiable AI: A Schema-Driven Platform for Safe and Auditable LLM Agent Deployment

The Problem That Started This

Large language models generate code quickly. Whether they generate it *correctly* is a different question entirely, and one that the industry has largely declined to answer. The prevailing approach treats “it runs” as a proxy for “it works”—and the result, frankly, is an industrial-scale production of mediocre software. Not broken software. Mediocre software, produced at volume, magnifying the quality problems already present in human-written code.

Six months of full-time research and development—building, testing, and rebuilding LLM-assisted coding systems across dozens of independent implementations—has made the structural nature of this problem clear. The failures are not random. They cluster predictably at the boundaries between application logic and the outside world: I/O operations, type safety at data import points, validation, and error handling. Roughly ninety percent of persistent errors fall into these four categories, a pattern that replicated consistently across six independently constructed applications. The clustering is not a prompting failure or a tooling gap. It is a training artifact—LLMs learned to write code from human code, and humans struggle at exactly these boundaries. The models inherit and amplify the difficulty.

Conventional responses—better prompts, stricter instructions, more tests—do not solve structural problems. Instructions fade as sessions extend. Tests generated by the same LLM that wrote the code share its assumptions and confirm its errors. Quality degrades toward the statistical centre of the training data, which is overwhelmingly mediocre. These are not harder versions of familiar software engineering problems. They are architectural problems that demand architectural responses.

The Insight: Schemas as the Universal Primitive

The solution that emerged was not a better way to prompt or a smarter linter. It was a recognition that schemas—structured, declarative specifications of behaviour, permissions, and constraints—could serve as the single source of truth from which everything else derives.

This insight began with code quality and kept generalising.

Verifiable code generation. A schema language defines what a piece of software should do: its inputs, outputs, permitted operations, and possible errors. From that schema, two independent generation tracks produce the implementation code and the test suite separately. Neither derives from the other. Both derive from the schema. This breaks the circular validation that plagues LLM-generated tests, where the same misunderstanding produces both the code and the tests that incorrectly confirm it. The schema is compact enough for a human to verify directly. The generated code is verifiable against the schema mechanically. The result is code whose correctness can be *proven* rather than merely hoped for.

The code generator is language-agnostic: the same schema produces implementations in Rust, Python, or other target languages through deterministic generation. A working demonstration exists.

Guardrail enforcement. The Gleipnir system uses schemas to define code quality constraints and enforces them as a pressure gradient rather than a pass/fail gate. Low-level violations require trivial fixes; high-level violations signal architectural problems demanding genuine restructuring. The system provides directional feedback rather than precise error descriptions, forcing the LLM to solve problems rather than game metrics. Over time, patterns that pass freely become the path of least resistance.

Boundary elimination. Rather than attempting to improve how LLMs handle the I/O boundaries where errors concentrate, the sandbox model eliminates those boundaries entirely. Application code lives inside a sandboxed environment with typed input and output channels. It cannot perform I/O directly—not because it has been instructed not to, but because the mechanism for doing so does not exist. The architecture makes the dominant error category structurally impossible.

The Deeper Problem: Vendor Dependency

Building this infrastructure revealed a second, more fundamental problem. Every tool, hook, and integration point that depends on a specific vendor's framework is fragile by nature. Vendors change behaviour without notice. Failures are silent—not error messages, but quietly degraded output that must be diagnosed backward from symptoms. A recent example: a vendor silently began summarising system prompt injections over ten thousand characters rather than delivering them in full. The LLM received a summary and a file link instead of the actual data. It ignored the file. An entire day of work was lost before the cause was identified.

This is not a complaint about any particular vendor. It is a structural observation. Any solution that depends on a vendor's hooks firing correctly, on framework internals remaining stable, or on the delivery chain passing data faithfully, is inherently fragile. The more capable and autonomous LLM agents become, the more consequential these silent failures are.

Bifrost: A Zero-Trust Proxy for LLM Traffic

The response is to move security, monitoring, and enforcement into the one layer the user fully controls: the traffic layer between the application and the LLM API.

Bifrost is a proxy that intercepts all LLM traffic in both directions. It is the only point in the stack that cannot be silently altered by a vendor update, and it is the only point with full visibility into what is actually being sent and received.

Its core capabilities:

- **Schema-enforced permissions.** What the LLM is permitted to do is defined through strict schemas. The proxy enforces those permissions on every request and response. Permissions are derived mechanically from the task definition, not configured manually.
- **Intention interception.** The proxy reads what the LLM intends to do *before* execution reaches the host system. Actions that fall outside permitted schemas are blocked before they can cause harm.
- **Behavioural monitoring.** Degradation signals—instruction drift, context exhaustion, anomalous patterns—are detected in the traffic stream, enabling early warning and automatic session termination before damage compounds.
- **Turn-by-turn context management.** Critical data is injected into each turn fresh and stripped afterward, ensuring that instructions never fade from context and that the context window stays clean.
- **Local sentinel model.** A small language model running on local hardware (independent of the primary LLM and its vendor) evaluates traffic against schemas and makes go/no-go decisions in real time. Configurable confidence thresholds allow users to set their own risk tolerance. When in doubt, the system shuts down. A failed task is always preferable to compromised data.

Bifrost should be open source. A zero-trust security layer that asks users to trust its developer without inspection would contradict its own founding principle. The code should be auditable by anyone who depends on it.

Deterministic Agent Harness: Auditable AI for Regulated Industries

On top of Bifrost sits the commercial product: a deterministic, vendor-agnostic system for building, testing, and certifying LLM agents.

Agent definitions are written as schemas in a vendor-neutral format. Vendor-specific transformers export these definitions to the format each provider requires. This separation means that agent logic is defined once and deployed to any model, and that switching providers does not require rebuilding agent configurations.

Because agent definitions are deterministic and version-controlled, they can be systematically benchmarked. When a new model version is released, the existing agent configurations are tested against the benchmark suite. If performance has shifted, the system can generate variant configurations—identical except for specific parameter changes—and test them in parallel to identify the optimal configuration for the new model. The certified configuration is stored with its benchmark results as an auditable record.

This matters most in regulated industries—finance, healthcare, legal, government—where deploying AI agents requires demonstrable proof that a specific configuration was tested against specific standards and produced specific, documented results. The deterministic agent harness provides the lifecycle that compliance demands: define, test, certify, deploy, and re-certify when conditions change.

Why This, Why Now, Why Me

The AI industry is accelerating toward autonomous agents with broad system access and minimal safety infrastructure. The tools to make this safe should exist before the tools to make it powerful, not after. That ordering has not been followed.

I have spent the past six months working full-time on this problem—not as a theoretical exercise but as a practitioner building real systems through sustained LLM collaboration, eight to ten hours a day. The published research (three technical papers examining the structural quality problems in LLM-generated code and the infrastructure required for sustained collaboration) emerged from that practice. The tools—Gleipnir, the sandbox architecture, the schema-based code generator, and the early Bifrost implementation—are working artifacts, not plans.

What this work has produced is a deep, experience-grounded understanding of how LLMs fail, how those failures compound, and what architectural responses actually hold up under sustained pressure. The schema-driven approach is the thread that runs through everything: from verifiable code generation through permission enforcement to auditable agent deployment. It is a coherent architecture, built from the ground up against real failure modes.

I would welcome the opportunity to walk through the technical detail and discuss where this goes next.